



Department of Computer Science

CS-250: Data Structure and Algorithms

Class: BESE-14 A

Semester Project Report

SUBMITTED BY:

Abdul Rehman Tahir (464140)

Muneeb Ur Rehman Bukhari (464485)

Hassan Shahid (465315)

BESE-14 A

December 29, 2024

SUBMITTED TO:

Sir Faisal Shafait

Project Report

1 Introduction

This documentation covers the details of the project that we have made **"The Search Engine"**. The project involves the development of a search engine designed to efficiently retrieve documents from a large dataset of more than 190,000 Medium articles. In addition, the user can also add his own documents to the dataset and can then search for his articles. The search engine uses indexing techniques, including inverted and forward indices, along with lexicon to ensure fast and accurate search results. By utilizing these data structures, the system allows users to query documents and receive relevant results based on keywords in a fraction of a second.

The whole project is in [GitHub account](#) in [this repository](#) which is currently private, but will be made public soon.

2 Motivation and Objective

The motivation behind this project is to create an efficient and scalable search engine that can quickly retrieve relevant documents from a large collection of articles. The objective is to build a search engine capable of processing complex queries, indexing a large dataset, and returning highly relevant results in real-time.

This project is inspired by Google's search engine and is an attempt to make a small scale engine similar to Google. For this, we first studied the smaller version of Google's research paper named **"The anatomy of a large-scale hyper-textual Web search engine"** and got an understanding of how Google actually works. Then picked a dataset and created a smaller replica of it. So, the main motivation and objective was to get an understanding of how search engines actually work, Google in particular, and others in general. In addition, we look at how data structures and algorithms play an essential role in increasing the performance and efficiency of the system.

3 Project Workflow

The whole project revolves around the use of data structures and algorithms. Creating indices for search engine is not a big deal but real work is in optimizing them to reduce latency as much as possible to give the user best possible experience. In general, our project workflow followed the streamline, shown in figure 1.

We will start discussing the detailed aspect of each step one by one:

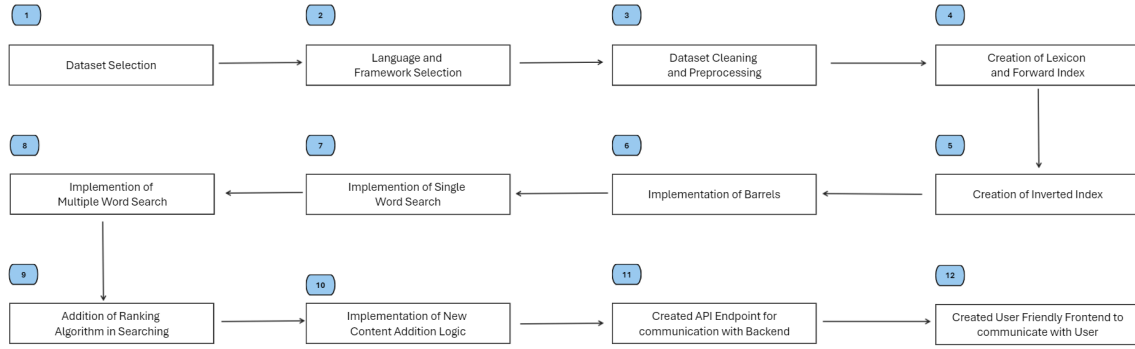


Figure 1: This is the workflow we followed while making the project.

3.1 Dataset Selection

The dataset we wanted was the dataset that should have fields which can help in searching and showing results, especially URL field. So, considering all this, we selected the “**190K+ Medium Articles**” which is a dataset containing medium articles and since the requirement was to chose a dataset more than 45,000 rows, this was perfect. It also has fields like text content, title of articles, publication date, tags and url that were helpful.

This data has been collected through a standard scraping process from the Medium website and the dataset has been downloaded from the [Kaggle](#).

Each row of this dataset contains information about a different medium article published. Following are the features provided for each of the article:

Field	Description
Title	The title of the article
Text	The text content of the article
URL	The URL associated to the article
Authors	The article authors
Timestamp	The publication datetime of the article
Tags	List of tags associated to the article

Table 1: Fields and Descriptions of the ”190K+ Medium Articles” dataset

3.2 Language and Framework Selection

The language and framework we chose were:

Python: Python was chosen for its simplicity and versatility, enabling efficient handling of data, implementing search algorithms, and backend development for the search engine. It also provides rich libraries for language processing such as **NLTK**.

Flask: Flask was used as the lightweight web framework to build the backend of the search engine, providing routing and server-side logic to interact with the frontend and search functionalities.

HTML, CSS, JavaScript: HTML structured the frontend layout, CSS provided styling and responsiveness, and JavaScript enabled dynamic user interactions, making the search engine interface intuitive and responsive.

3.3 Dataset Cleaning and Preprocessing

In the dataset cleaning, the dataset was loaded into the **Jupyter notebook** and was first checked for the missing values. Then, some insights were driven from the dataset with the help of graphs. At last, some new features were added in the dataset (feature engineering) which we thought will help in the searching process. The preprocessed dataset now contains 10 fields including the previous 6 and 4 new fields (text length, title length, number of tags, number of authors).

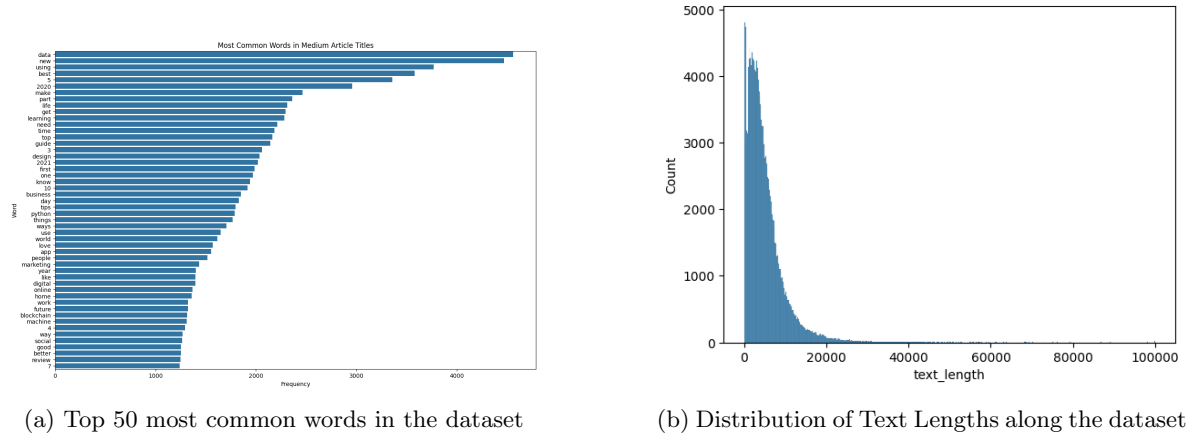


Figure 3: Getting Insights into Dataset

In the dataset preprocessing, the text and title content are separated and then **normalized** (converted to lower) for consistency. The content was done **tokenized** (separated individual words) and punctuation marks, numbers and stop words (common occurring words) are removed. From here on, **lemmatization** (converting words to their dictionary form) was done and POS (Parts of speech) tagging was used to ensure correct conversion. These techniques are all used via python's inbuilt library named **NLTK**. After this preprocessing, the content is ready for the creation of lexicon and forward index.

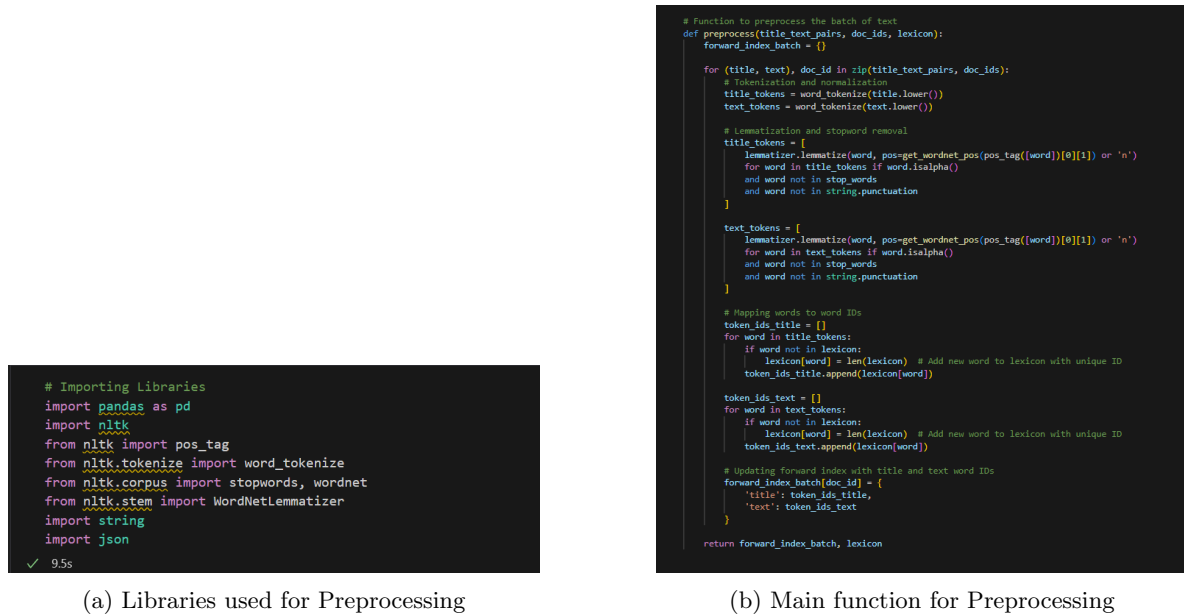


Figure 5: Some code snippets from Dataset Preprocessing

3.4 Creation of Lexicon and Forward Index

Lexicon: Lexicon is a collection of words or a list from the documents in the dataset. Lexicon helps in indexing the terms and making the fast retrieval of documents by performing term matching which helps in user query processing.

Forward Index: Forward index maps each document to the list of terms/words contained in the document, along with the optional metadata. In essence, the forward index tells “What terms are present in a particular document?”. The forward index actually helps in creating the inverted index (which maps the terms to the documents it appears in).

- The preprocessed text is taken and converted to a dictionary where each word is mapped with its word id (lexicon) and another dictionary where each document id is mapped with all the word ids it contains along with the position (title or text) where they appear. Since the dataset is pretty large, **batch processing** is being used where 5000 documents are being processed at one time. This reduces the time of creation of both lexicon and forward index. Both the lexicon and forward index were exported in a JSON File.

```
# Initializing the lexicon and forward index
lexicon = {}
forward_index = {}

batch_size = 5000
doc_id_prefix = "doc"

for i in range(0, len(articlesPreprocessed_dataset), batch_size):
    # Get batch texts and document ids (separate title and text)
    batch_titles = articlesPreprocessed_dataset["title"][i:i+batch_size]
    batch_texts = articlesPreprocessed_dataset["text"][i:i+batch_size]
    batch_texts_titles = list(zip(batch_titles, batch_texts))

    doc_ids = [f"{doc_id_prefix}{j}" for j in range(i, i + len(batch_texts_titles))]

    # Preprocessing the batch with document ids and lexicon
    batch_forward_index, lexicon = preprocess(batch_texts_titles, doc_ids, lexicon)

    # Updating the forward index and lexicon
    forward_index.update(batch_forward_index)
```

(a) Batch Processing for creation of Lexicon and Forward Index

Lexicon

```
{
  "mental": 0,
  "note": 1,
  "vol": 2,
  "photo": 3,
  "josh": 4,
  "riemer": 5,
  "unsplash": 6,
  "merry": 7
}
```

Forward Index

```
{"doc0":
  {"title": [0, 1, 2],
   "text": [3, 4, 5, 6, 7, 8, 9,
             10, 11, 12, 11, 13, 4]},
 "doc1":
  {"title": [313, 314],
   "text": [313, 314, 315,
             316, 317, 318, 319, 80]}}
```

(b) Example of Lexicon and forward index

Figure 7: Some code snippets from Lexicon and Forward Index creation

3.5 Creation of Inverted Index

Inverted Index: Inverted index is the opposite of the forward index. It maps each term to the document they appear in. In essence, it tells “Which documents contain the particular term?”. The inverted is actually the backbone of a search engine since it allows for a full-text search across large dataset and it also supports in advanced query handling such as phrase-search and proximity-based search.

- The hash table of forward index was loaded and it was used to make the inverted index. For each token (word id) in the forward index, the document id list (in which the word id appears in) is appended along with information such as document frequency, term frequency and position of word (title or text) with its frequency. So, it creates a structure of nested dictionaries. At last, the posting list is sorted for readability and then exported to JSON File.

Inverted Index

```
# Iterate through each document in the forward index
for doc_id, fields in forward_index.items():
    for field, word_ids in fields.items(): # 'field' can be 'title' or 'text'
        for word_id in word_ids: # Iterate over word IDs in each field
            if word_id not in inverted_index:
                inverted_index[word_id] = {'df': 0, 'postings': {}}
            if doc_id not in inverted_index[word_id]['postings']:
                inverted_index[word_id]['postings'][doc_id] = {'tf': 0, 'positions': {'title': 0, 'text': 0}}
                inverted_index[word_id]['df'] += 1
            # Increment term frequency and track positions in the respective field
            inverted_index[word_id]['postings'][doc_id]['tf'] += 1
            # Increment title/text position count
            inverted_index[word_id]['postings'][doc_id]['positions'][field] += 1
```

(a) Creation of Inverted Index

```
{0: {df: 2, postings: {doc0:
{tf: 3, positions: {title: 2,
text: 1}},
{doc1: {tf: 5, positions:
{title: 2, text: 3}}}}
```

(b) Example of Inverted Index

Figure 9: Some code snippets from Index creation

3.6 Implementation of Barrels

Barrels: Barrels are segmented data structures that divide the inverted index into smaller, manageable chunks based on specific criteria (e.g., word prefixes or ranges of word IDs). This segmentation improves efficiency by loading only relevant portions of the index into memory during a query.

- The inverted index was used to create barrels. Since the whole inverted index can't be loaded into memory at once, it must be divided so that the query can be processed. The criteria we used to divide the inverted index into barrels was based on the **word ids**. Each barrel will have a specified number of word ids so that they are small enough to be quickly loaded into memory. Our barrel structure is as follows:

- The first 10,000 words are divided into 1000 barrels each containing 10 words.
- The next 20,000 words are divided into 80 barrels each containing 250 words.
- The remaining words are divided into barrels such that each contains 10,000 words.

This structure is adopted so that each barrel is approximately equal to other and time taken to load them will be minimum during run time query processing.

```
# Processing and saving each barrel separately
for i in range(num_barrels):
    barrel = inverted_index_items[i * barrel_size : (i + 1) * barrel_size]
    barrel_dict = dict(barrel)

    # Saving the current barrel to JSON file
    print(f'Saving the barrel {i+1} with {len(barrel_dict)} word ids.')
    with open(f'Barrels/barrel_{i+1}.json', 'w') as f:
        json.dump(barrel_dict, f)

    print(f'Barrel {i+1} saved successfully.')
del barrel_dict
```

(a) Initial Barrel Creation

```
def get_barrel(word_id):
    # For barrel_10_1 to barrel_10_1000 (First 10000 words)
    if word_id < 10000:
        barrel_index = (word_id // 10) + 1
        return f'../Barrels/barrel_10/barrel_10_{barrel_index}.json'

    # For barrel_250_1 to barrel_250_80 (Next 20000 words)
    elif word_id >= 10000 and word_id < 30000:
        barrel_index = ((word_id - 10000) // 250) + 1
        return f'../Barrels/barrel_250/barrel_250_{barrel_index}.json'

    # For barrel_10000_1 to barrel_10000_46 (Remaining words)
    else:
        barrel_index = (word_id // 10000) - 2
        return f'../Barrels/barrel_10000/barrel_10000_{barrel_index}.json'
```

(b) Barrel Structure

Figure 11: Some code snippets from Barrels creation

3.7 Implementation of Single Word Search

Starting with the basic single word search algorithm, the process was straight forward. The input query of the user is normalized and tokenized, punctuation marks and stop words are removed and then the word is checked in lexicon for existence. If the word exists in lexicon, its word id is retrieved

and the corresponding barrel is loaded according to the word id. The word id is then matched in that barrel and the posting list (doc id list) for that word is shown.

We implemented **jaccard similarity** and **n-grams** for handling an edge case. What if the query user entered is not in the lexicon? Then we can't just show him empty results. What we do is we check for the closest word for the searched word that we have in the lexicon and then show the results for that word.

```
def single_word_search(query, lexicon):
    query = query.lower() # Normalizing the query

    if query not in lexicon:
        closest_match = get_closest_match(query, lexicon)
        print(f'No exact match found. Instead, showing results for {closest_match}')
        query = closest_match

    word_id = lexicon[query]
    print(f'The word id is {word_id}')
    if word_id is None:
        return f'Word '{query}' not found in the lexicon'

    barrel_file = get_barrel(word_id)
    if barrel_file is None:
        return f'Word '{query}' not found in the inverted index'

    try:
        with open(barrel_file, 'r') as f:
            barrel = json.load(f)
            print(f'barrel_file loaded.')

            if str(word_id) in barrel:
                return barrel[str(word_id)]["postings"]
            else:
                return f'Word '{query}' not found in the barrel file'
    except FileNotFoundError:
        return f'Barrel file '{barrel_file}' not found'

    return None
```

(a) Single word search function

```
def generate_ngrams(word, n=3):
    return [word[i:i+n] for i in range(len(word) - n + 1)]

def jaccard_similarity(grams1, grams2):
    intersection = set(grams1).intersection(set(grams2))
    union = set(grams1).union(set(grams2))
    return len(intersection) / len(union)

def get_closest_match(query, lexicon, n=3):
    query_ngrams = generate_ngrams(query, n)
    best_match = None
    best_score = 0

    for word in lexicon:
        word_ngrams = generate_ngrams(word, n)
        score = jaccard_similarity(query_ngrams, word_ngrams)

        if score > best_score:
            best_score = score
            best_match = word

    return best_match
```

(b) Closest word match using n-grams and Jaccard Similarity

Figure 13: Some code snippets from Single word search implementation

3.8 Implementation of Multiple Word Search

For handling the case where the user inputs the query consisting of multiple words, we first did the same things done in the single word search i.e, normalized the query and removed punctuation and stop words. Then the query is split to separate the words and for each word, the single word search function is called which returns the posting list of that word. Then afterwards, we just take the intersection of the posting list. In this way, we get those documents which contains all the words the user has entered.

For handling of an edge case where there is no document that has all the words of the query, we just show some documents that has only some of the words the user entered. In this way, user gets relevant results either way.

```
def multiple_word_search(query_string, lexicon=lexicon):
    original_query = query_string
    print(f'Query: {original_query}')

    # Removing punctuation marks from the query
    for char in query_string:
        if char in string.punctuation:
            query_string = query_string.replace(char, '') # Removing punctuation marks from the query

    query_words = query_string.lower().split() # Normalizing and then splitting the query
    if not query_words:
        return f'No result found for query: '{original_query}'\nQuery contains only punctuation marks.'"

    query_words = [word for word in query_words if word not in stopwords.words('english')]
    if not query_words:
        return f'Query '{original_query}' not found in the lexicon.'"

    print(f'Query words: {query_words}')

    # Getting the list of documents for each word in the query
    posting_list = []

    # Fetch posting list for each word in the query
    for word in query_words:
        postings = single_word_search(word, lexicon)

        if postings is not None and isinstance(postings, dict):
            posting_list.append(postings)

    if not posting_list:
        return f'No result found for query: '{original_query}'"
```

(a) Multiple word search function part 01

```
# Perform AND operation (intersection) on the posting lists
intersection_docs = set(posting_list[0].keys())
for postings in posting_list[1:]:
    intersection_docs.intersection_update(postings.keys())

# Collecting final postings from intersection
ranked_results = []
if intersection_docs:
    intersection_postings = {
        doc_id: postings[doc_id] for postings in posting_list for doc_id in intersection_docs
    }
    ranked_results = rank_documents(intersection_postings)

# Perform OR operation (union) on the posting lists if the result of intersection is very small
if len(intersection_docs) <= 3:
    union_docs = set()

    for posting in posting_list:
        union_docs.update(posting.keys())

    # Excluding intersection documents from the union
    union_docs.difference_update(intersection_docs)

    # Collecting final postings from union
    union_postings = {
        doc_id: postings[doc_id] for postings in posting_list for doc_id in union_docs if doc_id in postings
    }
    ranked_results.extend(rank_documents(union_postings))

return ranked_results[:15]
```

(b) Multiple word search function part 01

Figure 15: Some code snippets from Multiple word search implementation

3.9 Addition of Ranking Algorithm in Searching

Up till this point, the searching algorithm provides the results but there is no guarantee that the results are even relevant. For this, we implemented a simple yet elegant ranking algorithm. This algorithm works on the term frequency. Title appearance of the searched word has more weight than the text appearance. Using this formula, score is calculated for each document that come in results. Then sorting them in descending order, we showed the top 15 results that would be most relevant the entered query.

```
def calculate_score(posting):
    score = 0

    # Getting the term frequency from the title and the text of document
    tf_title = posting['positions'].get('title', 0)
    tf_text = posting['positions'].get('text', 0)

    # Calculating the score
    # Giving more weight to the title than the text
    score = (tf_title*10) + (tf_text*0.5)
    return score
```

(a) Calculating score function

```
def rank_documents(postings_list):
    ranked_docs = []

    for doc_id, posting in postings_list.items():
        score = calculate_score(posting)
        ranked_docs.append({'ID': doc_id, 'score': score})

    # Sorting the documents based on the score in descending order
    ranked_docs.sort(key=lambda x: x['score'], reverse=True)

    return ranked_docs
```

(b) Rank documents function

Figure 17: Some code snippets from Ranking Algorithm implementation

3.10 Implementation of New Content Addition Logic

The algorithm we used for the new content addition follows the following steps:

1. Take the user input according to the fields of the dataset.
2. Save the details in the new document csv file for record.
3. Append the new word of the document in the existing lexicon JSON file and create a new forward index JSON file for this document.
4. From the forward index, create the inverted index and then append the words in the respective barrels.
5. Once in barrel, the document can be searched.

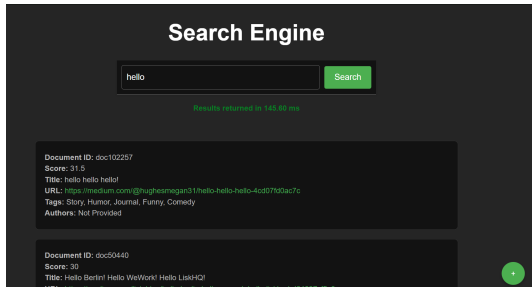
3.11 Creation of API Endpoint

After the completion of back-end logic, we integrated all the essential parts into a python script for the integration of the code with the front-end logic. **Flask** was used to create an API endpoint from where back-end and front-end can communicate with each other.

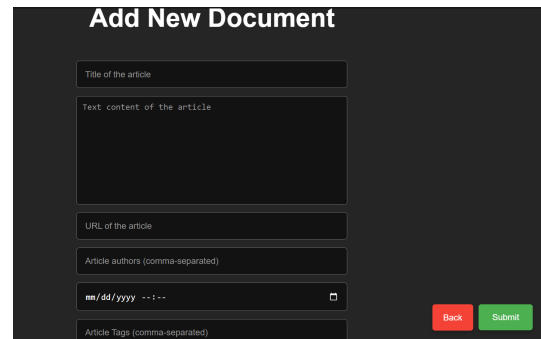
3.12 Creation of Front-End User Interface

The last thing done was the creation of the front-end for the user interaction. For this, we used flask templates in simple **HTML, CSS and Java Script** for the minimalistic user interface. The java script takes the input from the front end and give it to the API end point. From there, flask is responsible for the communication back and forth.

Our user interface consists of two pages, one for the searching and other for the addition of new content.



(a) The page from where user search



(b) The page from where user adds document

Figure 19: Some snippets from User Interface

4 Results

The search engine demonstrates impressive performance for standard queries, returning results in under a second. This showcases the efficiency of the indexing structures and search algorithms. However, for more complex or large queries, the response time can increase to 2–3 seconds due to the need for processing extensive data. While the query performance is within acceptable limits for most use cases, the document addition process requires significant optimization. Currently, uploading a document with approximately 1500 words takes around 5 minutes, which is not feasible for practical applications. This indicates a clear need for enhancements in the document addition pipeline to ensure real-time or near real-time updates to the index.

5 Conclusion

The project we made is not completely a success since there is much optimization that needed to be done. There are many other things that can be implemented in this search engine to make it even better. For example, we can update the inverted index to provide **phrase** and **proximity-based** search. Barrels structure can be improved. The document addition time can be reduced by directly creating inverted index without going to forward index. The ranking algorithm can be made to handle more cases ... and the list goes on.

In conclusion, there is still very much that can be done in this project but due to time constraint, we can't do it at the time. The current status of search engine is not very bad but it can surely be improved.